

实验四 差错控制

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 12 月 12 日

一、实验背景

实验目的：

1. 通过 Matlab 仿真，熟悉(7,4)汉明码的实现以及纠错性能的计算。
2. 通过(7,4)汉明码纠错实验，分析汉明码的纠错能力，熟悉汉明码的编码和译码原理，掌握生成矩阵和监督矩阵的生成方式。
3. 通过(7,4)汉明码交织实验，分析交织对于差错控制编码带来的性能提升，熟悉交织的原理和适用场景

Matlab 版本：2023b。

二、(7,4)汉明码的纠错实验

1. 实验步骤

- (1) 根据汉明码编译码的相关原理，完成汉明码的纠错过程。
- (2) 设置 BSC 信道误符号率 ε 分别为 0.001, 0.005, 0.01, 0.05, 0.1, 0.2，计算出不使用汉明码编码时的误比特率和误块率，以及使用汉明码编码时的误比特率和误块率。（将 4 个比特看做一个数据块，一个数据块中有一个比特错误则该块错误，以此来计算误块率）
- (3) 绘制随信道误符号率变化的误码率曲线，四条曲线画在同一张图上，有/无编码采用不同的颜色，误比特率/误块率采用不同的数据点符号样式，信道误符号率取值 $[1e-3, 2e-1]$ ，横纵坐标均采用对数坐标。
- (4) 观察绘制的曲线图，分析变化原因。（注意：特别关注在信道误符号率较低时，有/无编码对应的曲线斜率变化）

2. 代码实现

```
clear; close all; clc;
rng(2025);

% 码长
n = 7;

% 信息位长
k = 4;

% 传输比特块数
M = 10000;
```

```
% 生成矩阵
Q = [1 1 1; 1 1 0; 1 0 1; 0 1 1];
G = [eye(k) Q];

% 监督矩阵
H = [Q' eye(n-k)];

% 随机生成 M 个需要传输的比特块
x_data = randi([0 1], M, k);

% 将每个比特块编码成 (7, 4) 汉明码
x_code = mod(x_data*G, 2); % 进行编码 c=xG

% p_values = [0.001, 0.005, 0.01, 0.05, 0.1, 0.2];
p_values = logspace(-3, log10(0.2), 50);
num_p = length(p_values);
result = zeros(num_p, 4);

for j = 1:num_p
    % 经过误符号率为 p 的 BSC 信道
    p = p_values(j);
    noise = rand(M, n) < p;
    y = mod(x_code + noise, 2);

    % 利用监督矩阵计算校正子
    syndrome = mod(y * H', 2); % s=yH^T

    % 比较校正子和监督矩阵, 找出错误位置
    error_positions = zeros(M,1);
    H_transpose = H'; % 预计算 H'
    for i = 1:M
        if ismember(H',syndrome(i,:), 'rows') == zeros(n,1)
            error_positions(i,1) = 0;
        else
            error_positions(i,1) =
find(ismember(H',syndrome(i,:), 'rows'));
        end
    end

    % 进行纠错
    y_decode = y;
    for i = 1:M
        if error_positions(i,1) ~= 0
```

```
        y_decode(i, error_positions(i,1)) = ~y_decode(i,
error_positions(i,1));
    end
end

% 去掉监督位
y_decode_info = y_decode(:,1:k);

% 计算无信道编码时的误块率和误比特率
result_uncode = mod(x_data+y(:,1:k),2);
BitErrorRate_uncode = sum(result_uncode,'all')/(M*k);
BlockErrorRate_uncode =
sum(~ismember(result_uncode,zeros(1,k),'rows'))/M;

% 计算有信道编码时的误块率和误比特率
result_code = mod(x_data+y_decode_info, 2);
BitErrorRate_code = sum(result_code,'all')/(M*k);
BlockErrorRate_code = sum(~ismember(result_code, zeros(1,k),
'rows'))/M;

% 存储结果
result(j, :) = [BitErrorRate_uncode, BlockErrorRate_uncode,
BitErrorRate_code, BlockErrorRate_code];

% 输出误码率
% fprintf('BSC 信道误符号率: %.4f\n', p);
% fprintf('无汉明码: 误比特率 %.6f, 误块率 %.6f\n',
BitErrorRate_uncode, BlockErrorRate_uncode);
% fprintf('有汉明码: 误比特率 %.6f, 误块率 %.6f\n', BitErrorRate_code,
BlockErrorRate_code);
end

figure;
loglog(p_values, result(:, 1), 'bo-', 'MarkerSize', 3,
'MarkerFaceColor', 'b', 'DisplayName', '无汉明码-误比特率');
hold on;
loglog(p_values, result(:, 2), 'bo--', 'MarkerSize', 3,
'MarkerFaceColor', 'b', 'DisplayName', '无汉明码-误块率');
loglog(p_values, result(:, 3), 'ro-', 'MarkerSize', 3,
'MarkerFaceColor', 'r', 'DisplayName', '有汉明码-误比特率');
loglog(p_values, result(:, 4), 'ro--', 'MarkerSize', 3,
'MarkerFaceColor', 'r', 'DisplayName', '有汉明码-误块率');

grid on;
```

```

title('(7, 4)汉明码在 BSC 信道下的误码率性能');
xlabel('信道误符号率 \epsilon (对数坐标)');
ylabel('误码率 (对数坐标)');
legend('Location', 'southwest');
xlim([min(p_values) max(p_values)]); % 确保横坐标范围正确显示
hold off;

```

3. 实验数据记录及结果分析

(1) 设置 BSC 信道误符号率 ϵ 分别为 0.001, 0.005, 0.01, 0.05, 0.1, 0.2 计算出不使用汉明码编码时的误比特率和误块率, 以及使用汉明码编码时的误比特率和误块率

信道误符号率		0.001	0.005	0.01	0.05	0.1	0.2
无汉明码 编译码	数据块数	10000					
	误比特率	0.001225	0.004600	0.009975	0.050225	0.100450	0.198975
	误块率	0.04900	0.018300	0.039600	0.186800	0.342400	0.585000
有汉明码 编译码	数据块数	10000					
	误比特率	0.000000	0.000175	0.000525	0.019425	0.068800	0.196475
	误块率	0.000000	0.000500	0.001600	0.045800	0.151300	0.425400

表 1 差错控制编码实验记录

(2) 绘制随信道误符号率变化的误码率曲线

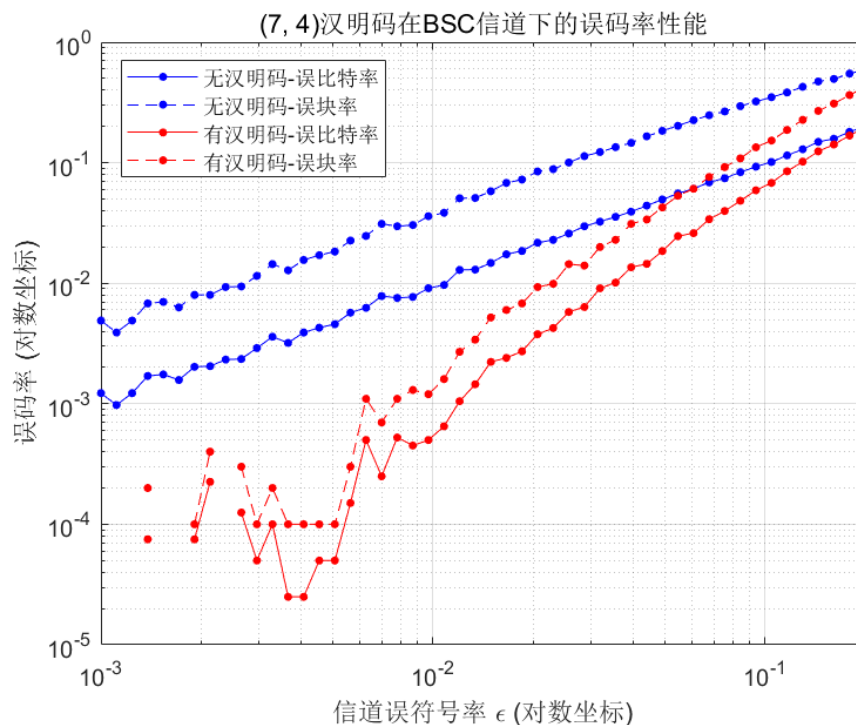


图 1 误码率随信道误符号率的变化曲线

(3) 分析曲线变化原因

由图可知，无汉明码情况下，误比特率和误块率稳定上升；有汉明码情况下，信道误符号率较低时误比特率和误码率能保持在较低处，但随着信道误符号率增长，误比特率和误块率也逐渐上升，且最终无汉明码和有汉明码情况下的误比特率和误码率十分接近。

推测原因可能是：在信道误符号率较低时，(7,4)汉明码可以有效纠正一比特错误，故在使用汉明码编译的情况下可以正确获得原码；但随着误码率不断上升，一位纠错已无法满足需求，因此误码率不断上升。

三、(7,4)汉明码的交织实验

1. 实验步骤

(1) 调试交织函数 `interleaver` 和解交织函数 `deinterleave`。

- ① 产生一个[1,2,3,...,35]的整数序列，使用 5 行 7 列的交织块，不编码，不经过信道，绘制交织前后，及解交织前后的序列，检查交织效果。
- ② 将①中序列换成长度为 35 的全零序列，并使交织后的序列经过一个信道传输，该信道在一个交织块内存在一个长度为 L 的突发错误，且起点随机。令 L 为交织块的行数，绘制并对比解交织前后的错误图案。(提示：调用 `burst_error` 函数产生突发错)
- ③ 令 L 为交织块行数的 2 倍，重做②。

(2) 随机生成 20 个比特的信息序列，每 4 个比特为一组使用(7,4)汉明码进行差错控制编码，经过一个信道传输，该信道在一个交织块内存在一个长度为 5 的突发错误，且起点随机。

- ① 使用一个不同于二中的生成矩阵 G ，并记录在实验报告中。
- ② 对汉明码编码后的序列进行 5 行 7 列的交织，经过信道后进行解交织，再进行译码。输出原始的信息序列 x ，汉明码编码后的序列 x_code ，交织后的序列 $x_interleave$ ，经过信道传输后的序列 y ，解交织后的序列 $y_deinterleave$ ，以及纠错后的序列 y_decode ，将其记录在实验报告中，并分析交织的作用和效果。
- ③ 随机生成 10000 个数据块，其余条件同②，改变一个交织块内存在的突发错误长度分别为 3, 5, 10, 15, 20, 25，对比使用交织/解交织和不使用无交织/解交织的系统可靠性。

2. 代码实现

```
clear; close all; clc;
rng(2025);

% 码长
```

```
n = 7;

% 信息位长
k = 4;

% 传输比特块数
% M = 5; % 20bit 传输时
M = 10000;

% 交织块行数
row = 5;
% 交织块列数
column = 7;
% 交织行列总数
count = row * column;
% 交织块数量
number = (M*n)/(row*column);

%% 验证交织解交织功能 1
ori_sequence1 = 1:count;
in_sequence1 = interleaver(row, column, ori_sequence1);
de_sequence1 = deinterleaver(row, column, in_sequence1);

figure;
subplot(3, 1, 1);
stem(ori_sequence1);
title('交织前序列');
xlabel('n');
ylabel('data');
subplot(3, 1, 2);
stem(in_sequence1);
title('交织后序列/解交织前序列');
xlabel('n');
ylabel('data');
subplot(3, 1, 3);
stem(de_sequence1);
title('解交织后序列');
xlabel('n');
ylabel('data');

%% 验证交织解交织功能 2
L = row;
ori_sequence2 = zeros(1, count);
in_sequence2 = interleaver(row, column, ori_sequence2);
```

```
in_error2 = burst_error(in_sequence2, L);
de_error2 = deinterleaver(row, column, in_error2);

figure;
subplot(3, 1, 1);
stem(in_sequence2);
title('交织后序列');
xlabel('n');
ylabel('data');
subplot(3, 1, 2);
stem(in_error2);
title('解交织前序列');
xlabel('n');
ylabel('data');
subplot(3, 1, 3);
stem(de_error2);
title('解交织后序列');
xlabel('n');
ylabel('data');

%% 验证交织解交织功能 3
L = 2 * row;
ori_sequence3 = zeros(1, count);
in_sequence3 = interleaver(row, column, ori_sequence3);
in_error3 = burst_error(in_sequence3, L);
de_error3 = deinterleaver(row, column, in_error3);

figure;
subplot(3, 1, 1);
stem(in_sequence3);
title('交织后序列');
xlabel('n');
ylabel('data');
subplot(3, 1, 2);
stem(in_error3);
title('解交织前序列');
xlabel('n');
ylabel('data');
subplot(3, 1, 3);
stem(de_error3);
title('解交织后序列');
xlabel('n');
ylabel('data');
```



```
%% 矩阵定义
% 生成矩阵
Q = [1 1 0; 1 0 1; 0 1 1; 1 1 1];
G = [eye(k) Q];
% 监督矩阵
H = [Q' eye(n-k)];

%% 有交织/解交织
% 随机生成 Mk 个需要传输的比特
x_data = randi([0 1], 1, M*k);
% fprintf('x_data\n');
% disp(x_data);

% 每 4 个信息比特作为一个比特块，编码成 (7, 4) 汉明码，得到比特流向量
x_code = zeros(1, M*n);
for i = 1:M
    x_code((i-1)*n+1:i*n) = mod(x_data((i-1)*k+1:i*k)*G, 2);
end
% fprintf('x_code\n');
% disp(x_code);

% 交织 按行写入 按列读出成比特流向量
x_interleave = zeros(1,M*n);
for i = 1:number
    x_interleave((i-1)*row*column+1:i*row*column) = interleaver(row,
column, x_code((i-1)*row*column+1:i*row*column));
end
% fprintf('x_interleave\n');
% disp(x_interleave);

% 经过信道 产生长度为 L 的突发错误
L = 25; % L = [3,5,10,15,20,25]
y = zeros(1,M*n);
for i = 1:number
    y((i-1)*row*column+1:i*row*column) = burst_error(x_interleave((i-1)*row*column+1:i*row*column),L);
end
% fprintf('y\n');
% disp(y);

% 解交织 按列写入 按行读出成比特流向量
y_deinterleave = zeros(1,M*n);
for i = 1:number
```

```
        y_deinterleave((i-1)*row*column+1:i*row*column)
    = deinterleaver(row, column, y((i-1)*row*column+1:i*row*column));
end
% fprintf('y_deinterleave\n');
% disp(y_deinterleave);

% 每 7 个比特为一组，进行译码
y_decode = zeros(1,M*k);
for i = 1:M
    % 利用监督矩阵计算校正子
    r = y_deinterleave((i-1)*n+1:i*n);
    syndrome = mod(r * H', 2);

    % 比较校正子和监督矩阵，找出错误位置
    if ismember(H',syndrome,'rows') == zeros(n,1)
        error_positions = 0;
    else
        error_positions = find(ismember(H',syndrome,'rows'));
    end

    % 进行纠错
    y_decode_all = y_deinterleave((i-1)*n+1:i*n);
    if error_positions ~= 0
        y_decode_all(error_positions) = ~y_decode_all(error_positions);
    end
    % 去除监督位
    y_decode((i-1)*k+1:i*k) = y_decode_all(1:k);
end
% fprintf('y_decode\n');
% disp(y_decode);

% 计算有交织时的误块率和误比特率
result = mod(x_data+y_decode,2);
result = transpose(reshape(result,[k,M]));
BlockErrorRate_code = sum(~ismember(result,zeros(1,k),'rows'))/M;
BitErrorRate_code = sum(result,'all')/(M*k);
fprintf('使用交织和(7,4)汉明码后的性能:\n');
fprintf('误比特率 %f\n', BitErrorRate_code);
fprintf('误块率 %f\n', BlockErrorRate_code);

%% 无交织/解交织
total_errors = number * L; % 总突发错误长度
y0 = x_code; % 初始化
```

```
% 将数据流分成若干段，每段添加一个突发错误
segment_length = floor(M*n / number); % 每段长度≈交织块大小
for i = 1:number
    start_idx = (i-1)*segment_length + 1;
    end_idx = min(i*segment_length, M*n);
    if end_idx - start_idx + 1 >= L % 确保段长度足够，在该段添加一个 L 长度
    的突发错误
        segment = y0(start_idx:end_idx); % 提取该段
        corrupted_segment = burst_error(segment, L); % 在该段添加突发错误
        y0(start_idx:end_idx) = corrupted_segment; % 替换回原数据流
    end
end

% 每 7 个比特为一组，进行译码
y_decode0 = zeros(1,M*k);
for i = 1:M
    % 利用监督矩阵计算校正子
    r = y0((i-1)*n+1:i*n);
    syndrome = mod(r * H', 2);

    % 比较校正子和监督矩阵，找出错误位置
    if ismember(H',syndrome,'rows') == zeros(n,1)
        error_positions = 0;
    else
        error_positions = find(ismember(H',syndrome,'rows'));
    end

    % 进行纠错
    y_decode_all0 = y0((i-1)*n+1:i*n);
    if error_positions ~= 0
        y_decode_all0(error_positions) =
~y_decode_all0(error_positions);
    end
    % 去除监督位
    y_decode0((i-1)*k+1:i*k) = y_decode_all0(1:k);
end
% fprintf('y_decode0\n');
% disp(y_decode0);

% 计算误码率
result0 = mod(x_data + y_decode0, 2);
result0 = transpose(reshape(result0,[k,M]));
BlockErrorRate_code0 = sum(~ismember(result0,zeros(1,k),'rows'))/M;
BitErrorRate_code0 = sum(result0,'all')/(M*k);
```

```
fprintf('无交织和(7,4)汉明码后的性能:\n');
fprintf('误比特率 %f\n', BitErrorRate_code0);
fprintf('误块率 %f\n', BlockErrorRate_code0);

%% 函数定义
function x_interleave = interleaver(row, column, x)
x = reshape(x, column, row);
x = x';
x_interleave = x(:)';
end

function y = burst_error(x, L)
noise = zeros(1,size(x,2));
error_idx = randi([1,size(x,2)-L+1]);
noise(1,error_idx:error_idx+L-1) = (rand(1,L)<0.5);
y = mod(x + noise, 2);
end

function y_deinterleave = deinterleaver(row, column, y)
y = reshape(y, row, column);
y = y';
y_deinterleave = y(:)';
end
```

3. 实验数据记录及结果分析

(1) 调试交织函数 interleaver 和解交织函数 deinterleaver

- ① 给出使用[1,2,3,...,35]的整数序列时，交织前后及解交织前后的序列

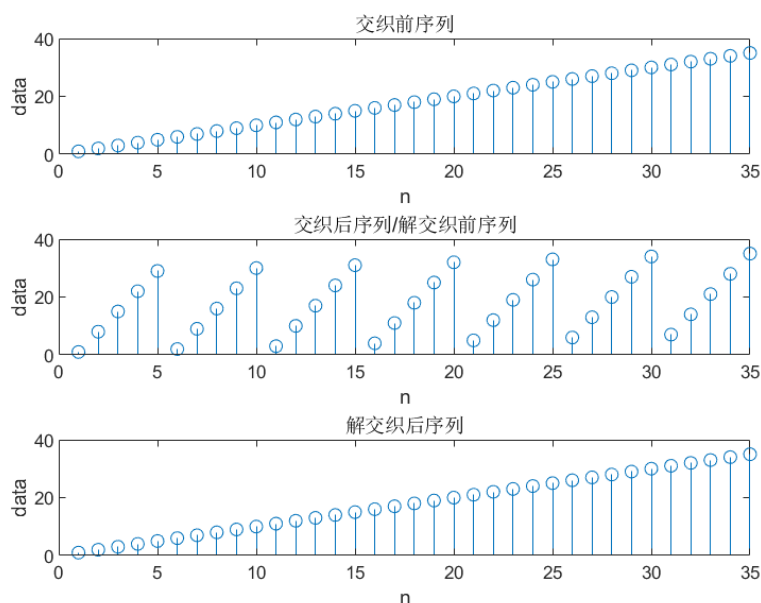


图 2 交织前后及解交织前后的序列

② 给出使用全零的整数序列，突发错误长度为交织块行数时，解交织前后的错误图案

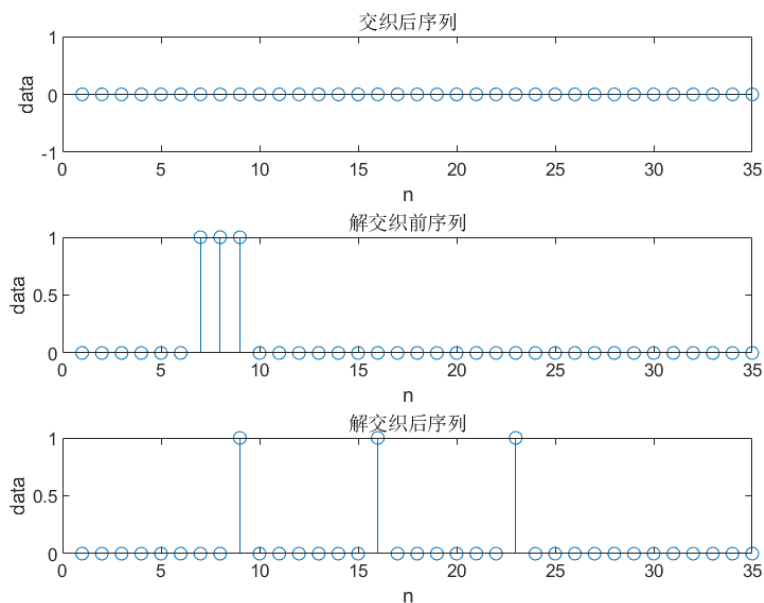


图 3 解交织前后的错误图案

③ 突发错误长度为交织块行数的 2 倍时，解交织前后的错误图案

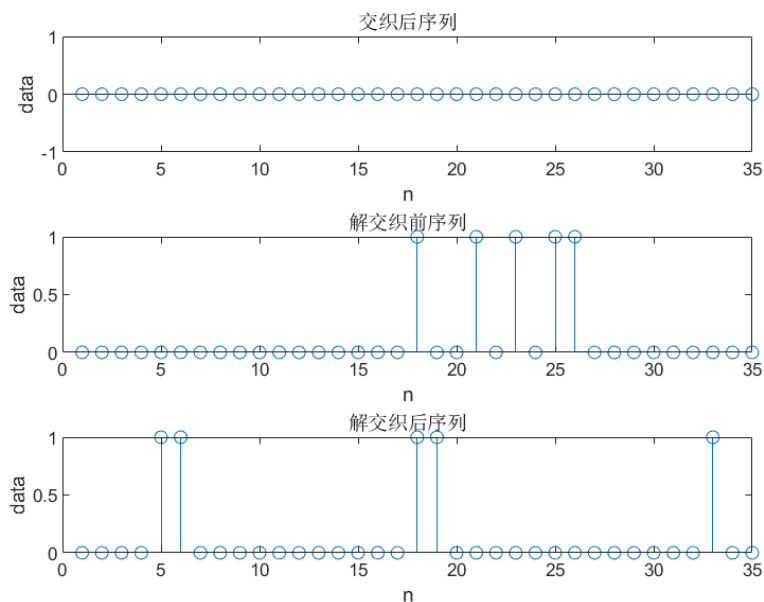


图 4 解交织前后的错误图案

由图可知，交织操作改变了原始序列的排列顺序，而解交织过程能够完全恢复原始序列的排列。当交织后的序列通过存在突发错误的信道时，会引入连续的突发错误。经过解交织

处理后，原本集中的连续错误被分散到不同的位置。当突发错误长度等于交织块的行数时，所有错误均被有效地分散为非连续错误；然而，当突发错误长度达到交织块行数的两倍时，部分错误仍可能以连续形式存在，未能被完全分散。

(2) 差错控制编码与交织

① 给出所使用的生成矩阵 G

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

② 给出原始的信息序列 x，汉明码编码后的序 x_code，交织后的序列 x_interleave，经过信道传输后的序列 y，解交织后的序 y_deinterleave，以及纠错后的序列 y_decode

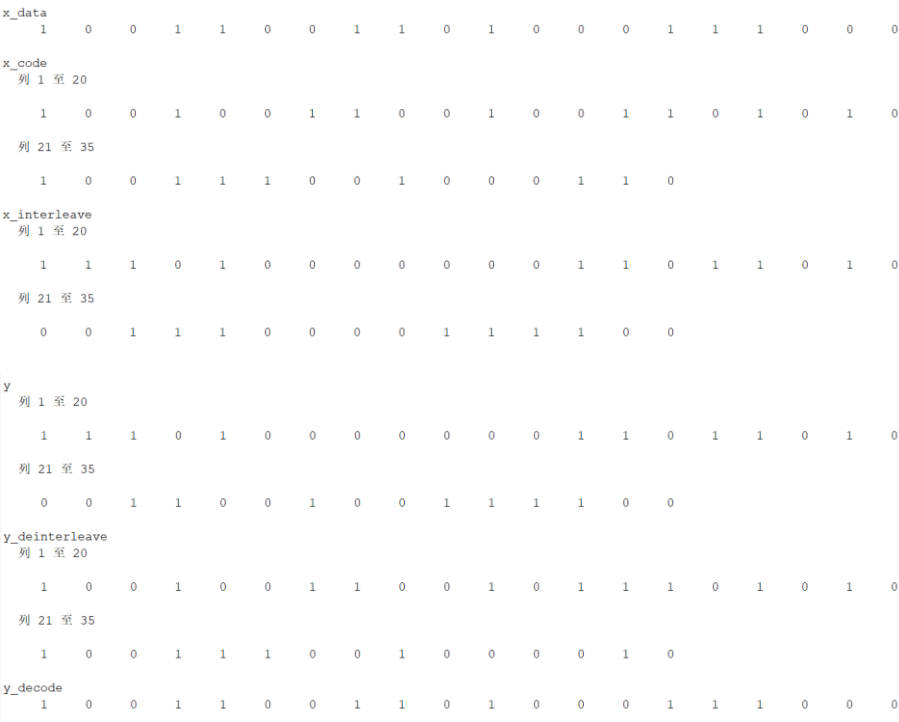


图 5 20 比特信息序列的解码全流程输出

x	10011001101000111000
x_code	10010011001001101010100111001000110
x_interleave	11101000000011011010001110000111100
y	11101000000011011010001100100111100
y_deinterleave	10010011001011101010100111001000010
y_decode	10011001101000111000

表 2 20 比特信息序列的解码全流程输出

通过实验可以发现，交织技术能够重新排列编码后的数据序列，将原本集中的突发错误分散到不同位置。由于汉明码只能纠正一比特错误，这种分散处理能够显著提升系统的整体纠错性能，有效抑制信道失真，从而提高数据传输的准确性和可靠性。

(3) 随机生成 10000 个数据块，其余条件同(2)，改变一个交织块内存在的突发错误长度分别为 3, 5, 10, 15, 20, 25，对比使用交织/解交织和不使用无交织/解交织的系统可靠性

信道突发错误长度		3	5	10	15	20	25
无交织/ 解交织	数据块数	10000					
	误比特率	0.046375	0.077275	0.148925	0.217650	0.292700	0.367175
	误块率	0.091900	0.155500	0.289000	0.419300	0.555700	0.701000
有交织/ 解交织	数据块数	10000					
	误比特率	0.000000	0.000000	0.120000	0.250500	0.344000	0.404825
	误块率	0.000000	0.000000	0.241500	0.502000	0.689400	0.809800

表 3 差错控制编码实验记录

当突发错误长度小于或等于交织块行数时，交织/解交织操作能够显著降低信号的误比特率和误块率。然而，随着突发错误长度的增加，交织/解交织对误比特率与误块率的改善作用几乎消失，甚至未采用交织处理的系统在误比特率与误块率上表现略优。可能原因是汉明码仅能纠正单个错误，当错误分散后，若多个码字各含一个错误，纠错负担并未减少；而解交织过程可能导致原本集中于单个码字的多个错误被分散至更多码字中，反而增加了多个码字发生错误的概率。

四、 实验总结

本次通信与网络课程实验通过 MATLAB 仿真研究了(7,4)汉明码的纠错与交织。实验首先实现了汉明码在 BSC 信道下的编译码过程，验证了其纠正单比特错误的能力。

仿真结果表明，在信道误符号率较低时，采用汉明码可显著降低误比特率和误块率，但随着信道误符号率增大，编码增益逐渐减小。实验进一步探究了交织技术对抗连续突发错误的效果，观察到交织操作能将连续的突发错误分散到不同码字中。当突发错误长度不超过交织块行数时，交织能充分发挥作用，大幅提升系统可靠性；但当错误长度超过交织块大小时，分散效果有限，性能改善不明显。

通过本次实验，我基本熟悉了汉明码的实现、纠错性能的计算，以及交织对于差错控制编码带来的性能提升。